

Session 08 Homework — Extend the DTOs

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

This extends B4F Board's `api/` exactly where class left off: both `ProjectsModule` and `WorkspacesModule` have real DTOs, a global `ValidationPipe`, and a validated `POST / PATCH` pair, built in class and in the teamwork.

Goal

Add an unrequired `description` field to both `Projects`' and `Workspaces`' DTOs, actually use it in both services, and strengthen the global `ValidationPipe` by one more option — using only decorators and options already used in class.

What to build

A `description?` field on `CreateProjectDto / UpdateProjectDto` (CORE), the identical field on `CreateWorkspaceDto / UpdateWorkspaceDto` (STRETCH), and `forbidNonWhitelisted: true` added to the existing `ValidationPipe` in `main.ts` (CHALLENGE).

CORE (required)

Add an unrequired `description` field to `CreateProjectDto` and `UpdateProjectDto`:

- `@IsOptional()` `@IsString()` — the same two decorators you already used for `name / key`. Do **not** add `@IsNotEmpty()` — clearing a description back to empty is a reasonable thing to want, unlike `name / key`.
- Update `ProjectsService.create()` to use `dto.description` (defaulting to `''` if not sent) instead of always hard-coding an empty string.
- Update `ProjectsService.update()` to apply `dto.description` when it was actually sent (check `!== undefined`, the same pattern you already used for `name / key`).

STRETCH (required)

Do the identical thing for `Workspaces`: `description?` on `CreateWorkspaceDto / UpdateWorkspaceDto`, used in `WorkspacesService.create() / update()`.

CHALLENGE (required)

In `main.ts`, add `forbidNonWhitelisted: true` as a second property on the SAME `ValidationPipe` object you already built in class (alongside `whitelist: true`) — not a second pipe.

Test the difference it makes: send a request with a field no DTO declares, before and after adding this option. Be ready to explain, in one sentence, what changed.

Rules

Allowed: everything taught in Session 08 — `@IsOptional()`, `@IsString()`, and one more property on the `ValidationPipe` object already built in class.

Not yet: no new decorators beyond the two already used, no guards, no database, no new npm package, no `@IsNotEmpty()` on `description` (see CORE above for why).

Test it

BASH

```
curl -X POST http://localhost:3000/projects -H "Content-Type: application/json" -d '{"name": "Docs Site", "key": "DOCS", "description": "Public documentation"}'
curl -X PATCH http://localhost:3000/projects/1 -H "Content-Type: application/json" -d '{"name": "Ok", "notAField": true}'
```

Done when

- A `POST / PATCH` with a `description` field actually stores and returns it, on both `Projects` and `Workspaces`.
- A request with an unexpected field name returns `400` naming that exact field (CHALLENGE done).
- `npm run start:dev` starts with no errors and no warnings in the terminal.
- You can explain, out loud, why `description` does not get `@IsNotEmpty()` when `name / key` do.

Working time

Expect roughly 25-35 minutes total across all three tiers.